

Image Restoration and Compression

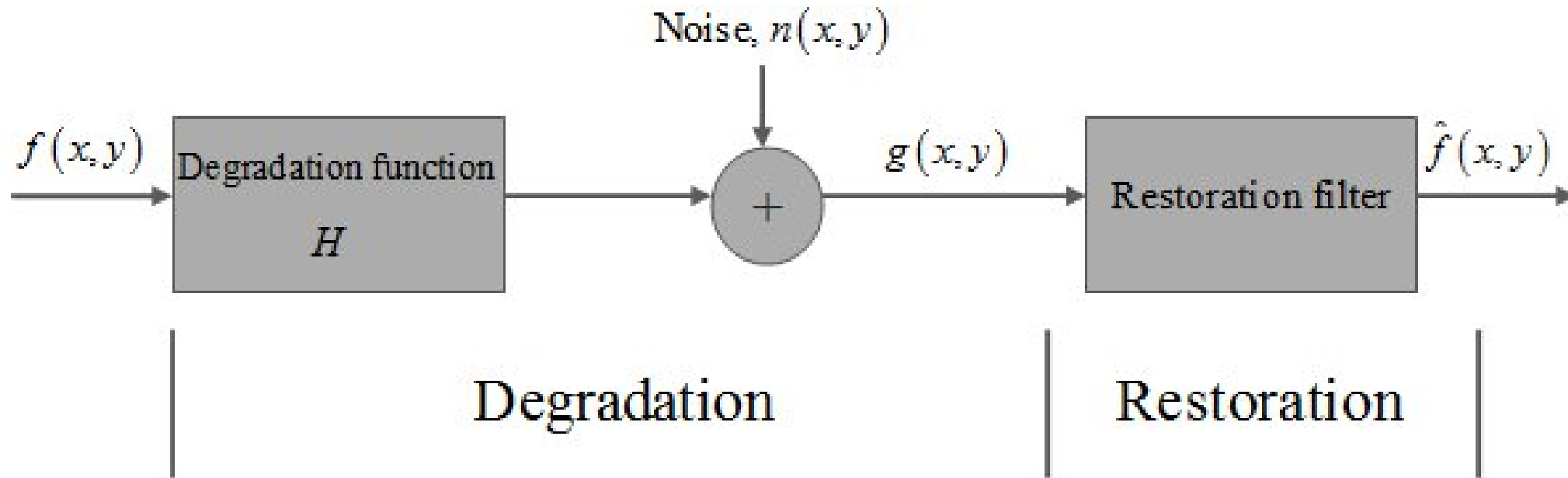
UNIT 4 | Yuba Raj Devkota

NCCS

Unit 4	Image Restoration and Compression	Teaching Hours (8)
Image Restoration	Introduction, Models for Image degradation and restoration process, Noise Models (Gaussian,	2 hrs.
	Rayleigh, Erlang, Exponential, Uniform and Impulse), Estimation of Noise Parameters	
Restoration Filters	Mean Filters: Arithmetic, Geometric, Harmonic and Contraharmonic Mean Filters Order Statistics Filters: Median, Min and Max, Midpoint and Alpha trimmed mean filters Band pass and Band Reject filters: Ideal, Butterworth and Gaussian Band pass and Band Reject filters	2 hrs.
Image Compression	Introduction, Definition of Compression Ratio, Relative Data Redundancy, Average Length of Code Redundancies in Image: Coding Redundancy (Huffman Coding), Interpixel Redundancy (Run Length Coding) and Psychovisual Redundancy (4-bit Improved Gray Scale Coding: IGS Coding Scheme)	2 hrs.
Image compression models:	Lossless and Lossy Predictive Model (Block Diagram and Explanation)	2 hrs.

What is Image Restoration?

- image restoration is an essential approach used for the retrieval of the uncorrupted, original images from blurred and noisy images because of motion blur, noise, etc. caused by environmental effects and camera misfocus.
- The technique of image restoration involves recovering an image from a damaged state—typically a distorted and noisy image. In image processing, image restoration is a core issue.
- Image restoration is done in both frequency domain and spatial domain.



$$g(x, y) = h(x, y) * f(x, y) + n(x, y) \rightarrow \textcircled{1}$$

$$G(u, v) = H(u, v) F(u, v) + N(u, v) \rightarrow \textcircled{2}$$



True image



Blurred and noisy image



Restoration by Algorithm 1



Restoration by Algorithm 3



True image



Blurred and noisy image



Restoration by Algorithm 1



Restoration by Algorithm 3



True image



Blurred and noisy image



Restoration by Algorithm 1



Restoration by Algorithm 3



True image



Blurred and noisy image

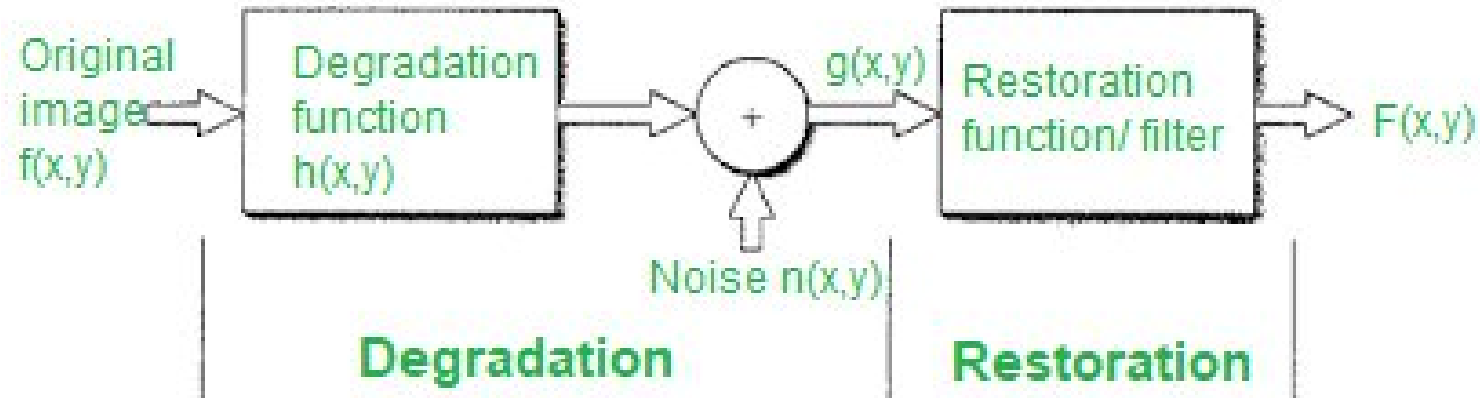


Restoration by Algorithm 1



Restoration by Algorithm 3

Noise Models in Digital Image Processing (Noise Probability Density Function, PDF)

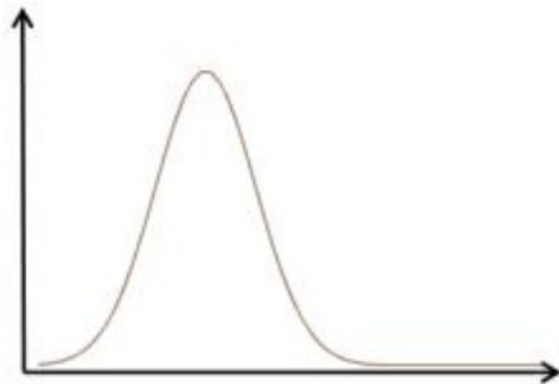


$$g(x, y) = h(x, y) * f(x, y) + n(x, y)$$

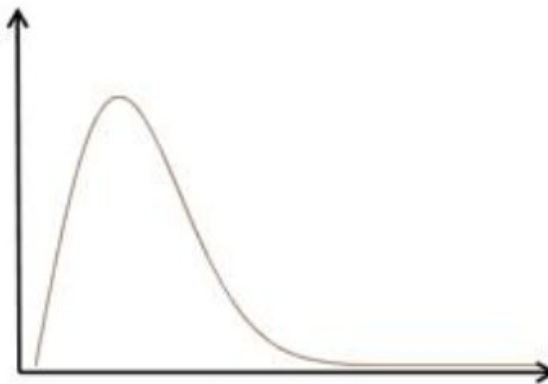
The goal of the restoration function or the restoration filter is to obtain a close replica $F(x,y)$ of the original image.

Different Noise Models

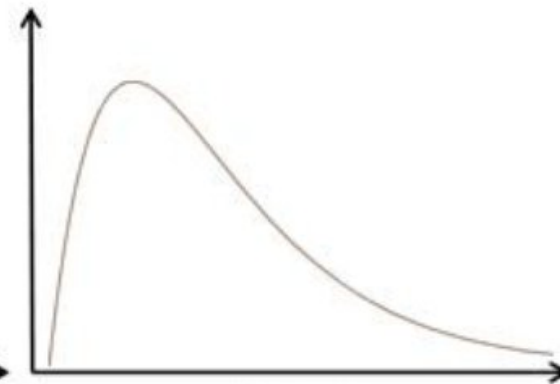
In image processing, a noise model mathematically describes how noise is introduced into an image during acquisition, transmission, or storage. Different physical situations produce different noise models.



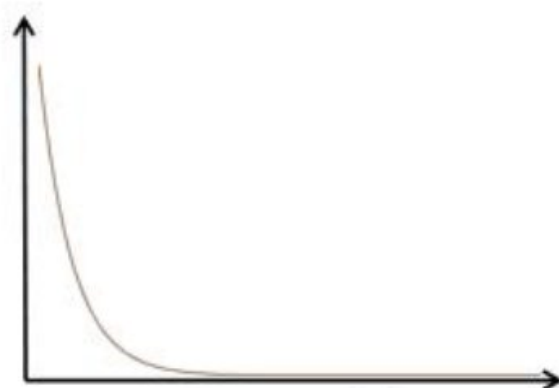
Gaussian



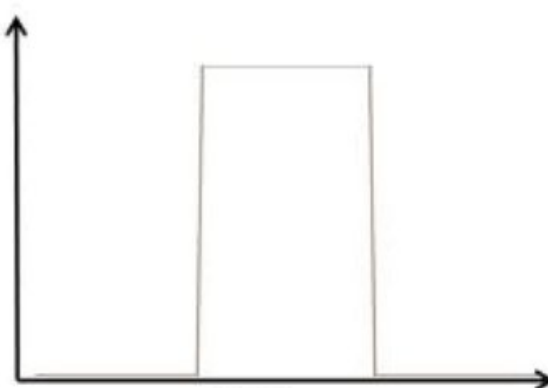
Rayleigh



Gamma



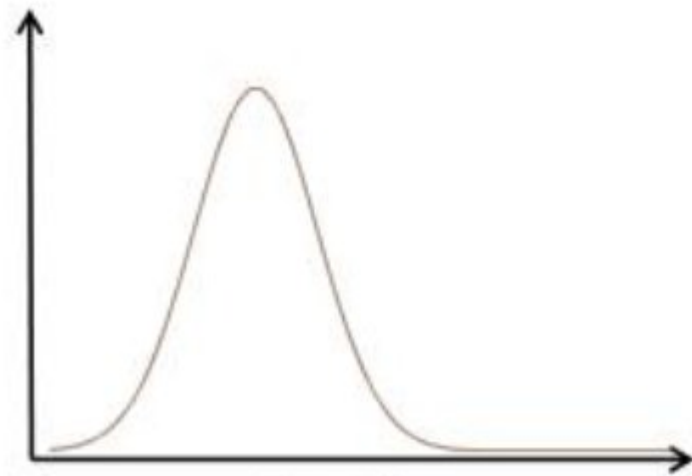
Exponential



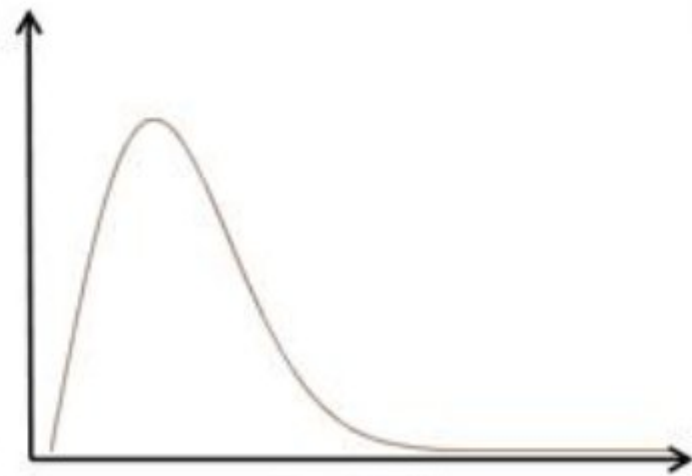
Uniform



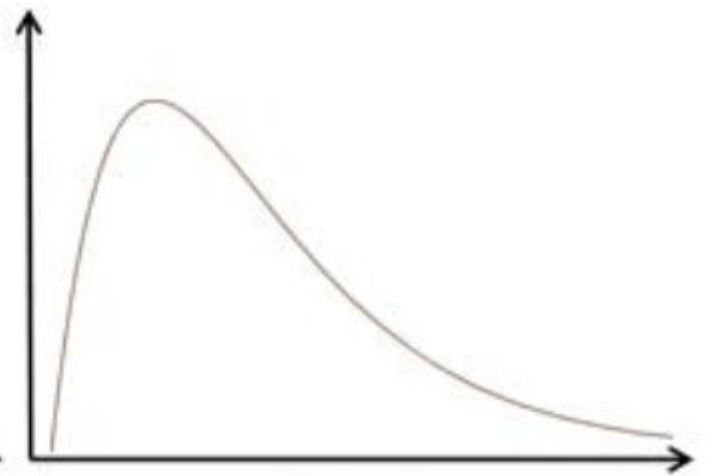
Impulse



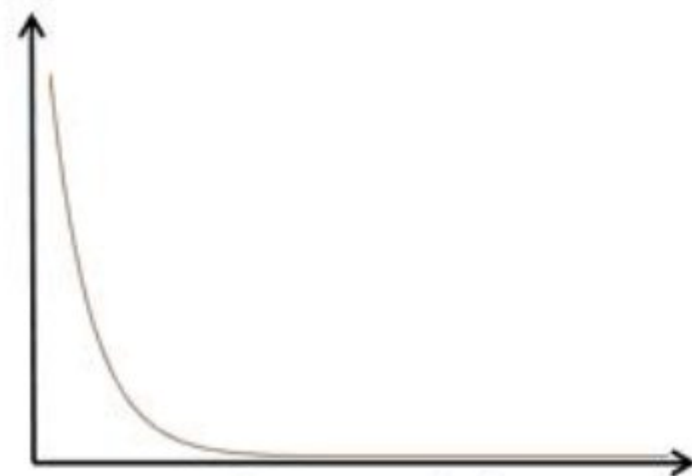
Gaussian



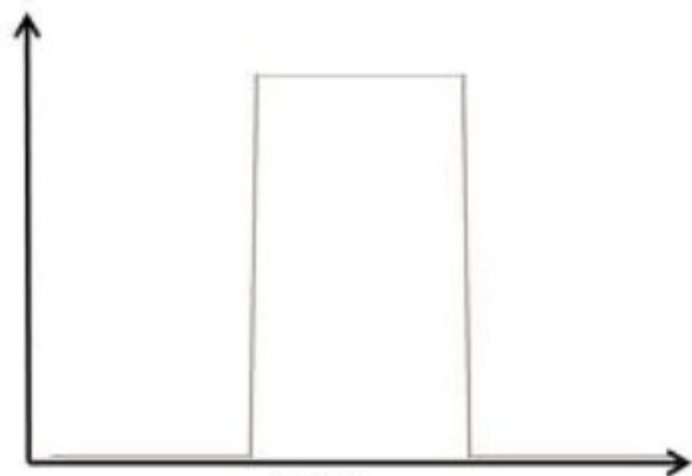
Rayleigh



Gamma



Exponential



Uniform



Impulse

1. Gaussian Noise (Additive White Gaussian Noise – AWGN)

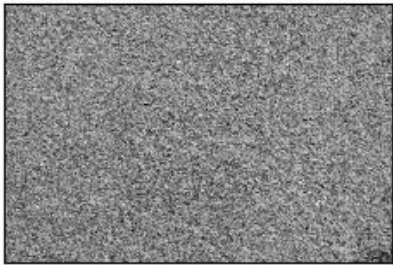
Model

$$g(x, y) = f(x, y) + n(x, y)$$

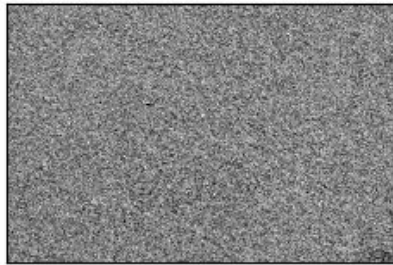
where $n(x, y)$ follows a **Gaussian (normal)** distribution:

$$p(n) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(n-\mu)^2}{2\sigma^2}}$$

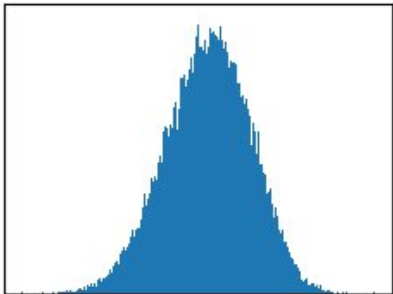
Original Image



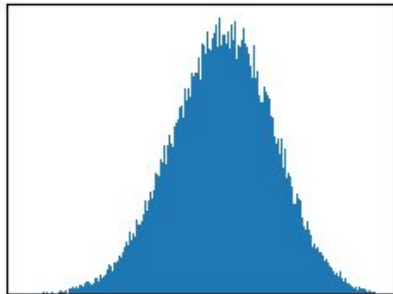
Noisy Image



Original Image Histogram



Noisy Image Histogram



Original Image



Gaussian Noise

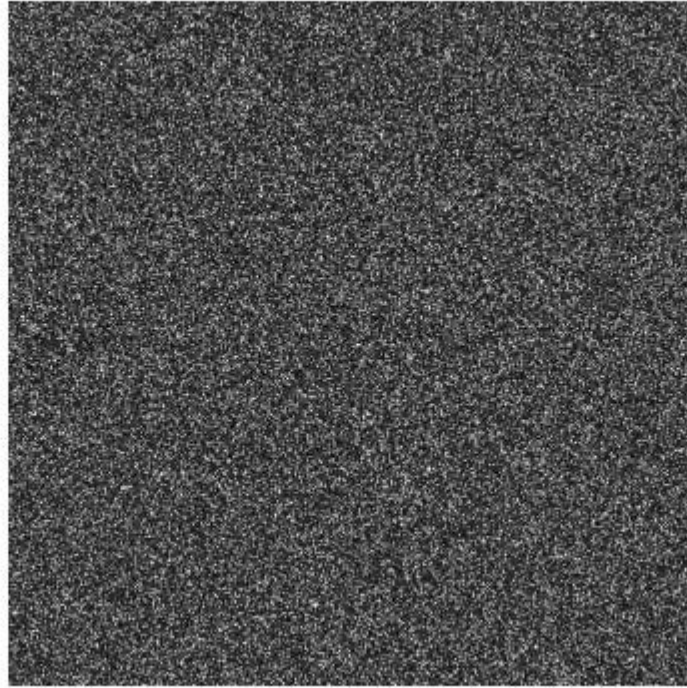


Image with Gaussian Noise



Characteristics

- Continuous noise
- Mean = μ , Variance = σ^2
- Affects **all pixels**
- Histogram is bell-shaped

Applications / Sources

- Sensor thermal noise
- Electronic circuit noise
- Low-light photography
- Satellite and medical imaging

2. Salt and Pepper Noise (Impulse Noise)

Model

$$p(z) = \begin{cases} P_a & z = a \text{ (pepper / black)} \\ P_b & z = b \text{ (salt / white)} \\ 0 & \text{otherwise} \end{cases}$$

Typically:

- $a = 0$
- $b = 255$





Characteristics

- Random black and white pixels
- Caused by sudden disturbances
- Discrete noise

Applications / Sources

- Faulty camera sensors
- Bit errors in transmission
- Dead pixels

✓ **Best filter:** Median Filter

3. Poisson Noise (Shot Noise)

Model

$$p(k) = \frac{\lambda^k e^{-\lambda}}{k!}$$

where:

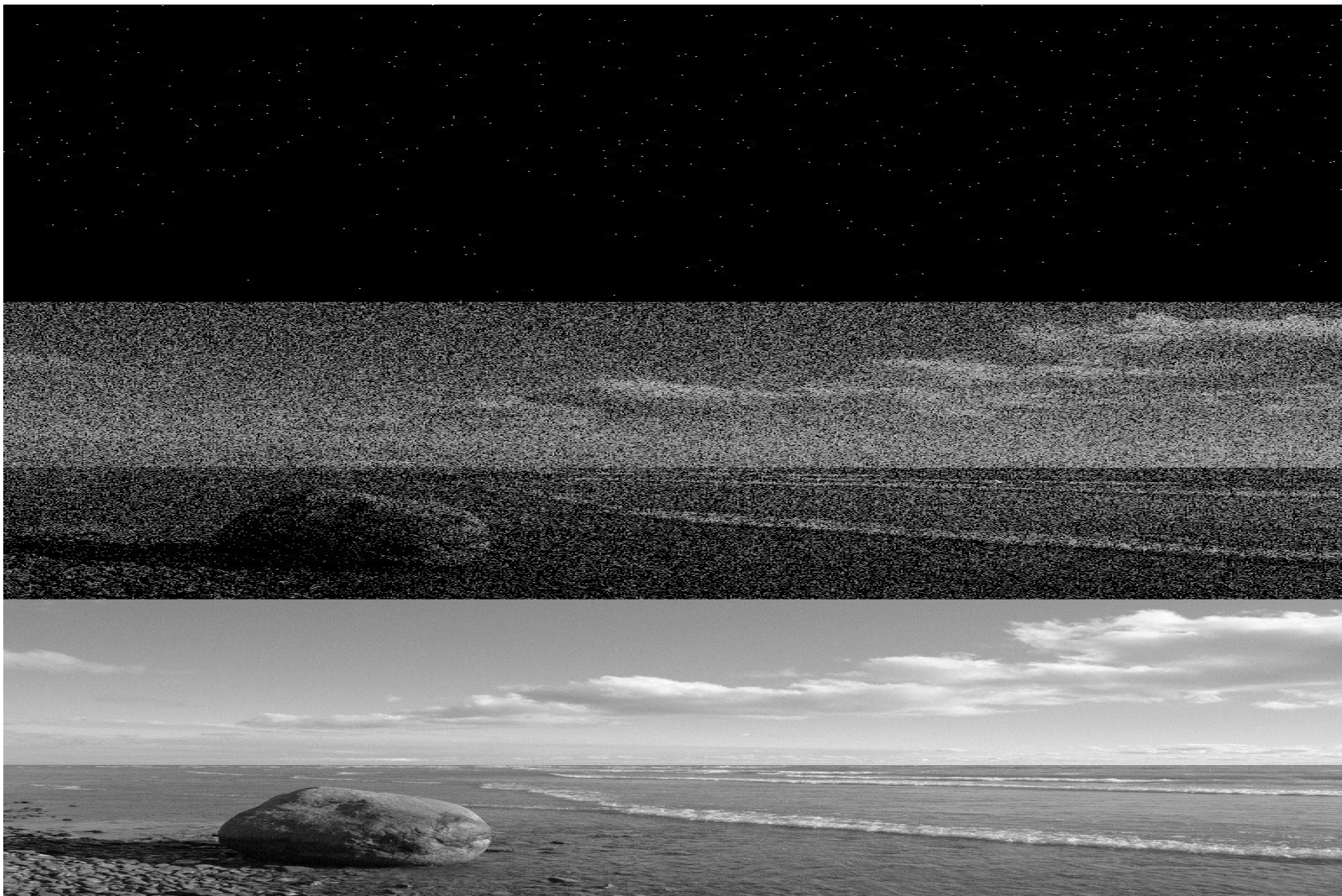
- λ = average number of photons
- k = observed photon count

Characteristics

- Signal-dependent noise
- Occurs when photon count is low
- Variance = Mean

Applications / Sources

- Medical imaging (X-ray, CT)
- Astronomy
- Low-light imaging



4. Speckle Noise (Multiplicative Noise)

Characteristics

- Multiplicative in nature
- Granular appearance
- Degrades edge information

Model

$$g(x, y) = f(x, y) \cdot n(x, y)$$

or

$$g(x, y) = f(x, y) + f(x, y) \cdot n(x, y)$$

Applications / Sources

- Ultrasound imaging
- Synthetic Aperture Radar (SAR)
- Laser imaging



Fig 1: Original Image



Speckle Noisy Image

5. Uniform Noise

Model

$$p(z) = \begin{cases} \frac{1}{b-a}, & a \leq z \leq b \\ 0, & \text{otherwise} \end{cases}$$

Characteristics

- Equal probability for all values
- Artificial noise (used in testing)

Applications

- Simulation and algorithm testing
- Performance analysis of filters

Original Image



Uniform Noise

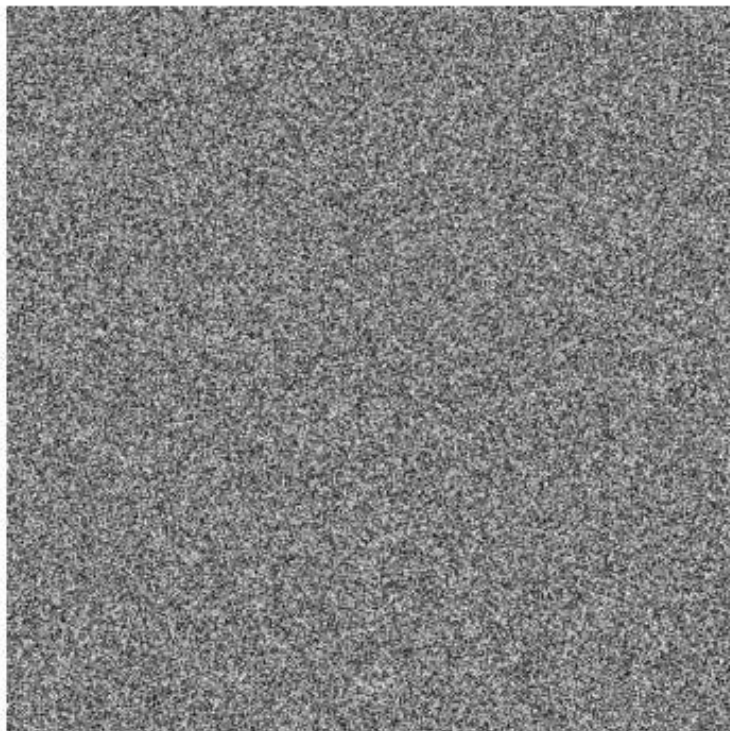


Image with Uniform Noise



6. Rayleigh Noise

Model

$$p(z) = \begin{cases} \frac{z}{\sigma^2} e^{-\frac{z^2}{2\sigma^2}}, & z \geq 0 \\ 0, & z < 0 \end{cases}$$

Characteristics

- Skewed distribution
- Used in modeling magnitude of noisy signals

Applications

- Radar imaging
- Wireless signal amplitude modeling



Gaussian noise



Rayleigh noise



Gamma noise



Exponential noise



Uniform noise



Impulse noise

Noise Model	Type	Formula Nature	Common Applications
Gaussian	Additive	Normal PDF	Sensors, cameras
Salt & Pepper	Impulse	Discrete	Transmission errors
Poisson	Signal-dependent	Photon counting	Medical, astronomy
Speckle	Multiplicative	Product model	SAR, ultrasound
Uniform	Additive	Constant PDF	Testing
Rayleigh	Statistical	Skewed PDF	Radar

Restoration in the presence of Noise Only (Spatial Filtering)

- The degradation image is given by:

$$g(x,y) = h(x,y)*f(x,y) + n(x,y).$$

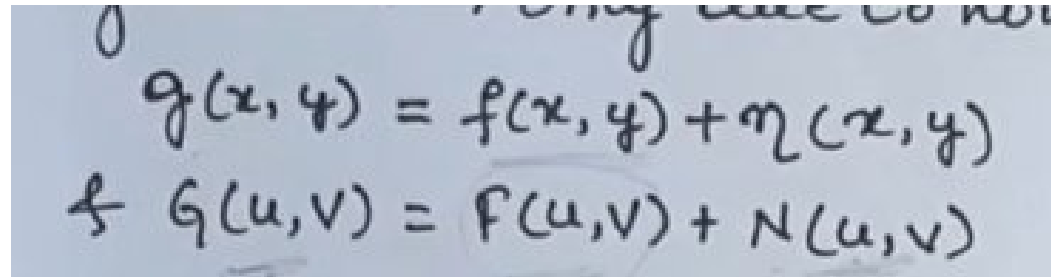
- Since degradation is only due to noise, we omit the part $h(x,y)$, so the new equation and its Fourier transform becomes as follows.
- The original image $F(u,v)$ can be obtained by subtracting noise from Frequency domain image i.e. $G(u,v) - N(u,v)$.

1. Mean Filters

1. Arithmetic Mean Filters
2. Geometric Mean Filters
3. Harmonic Mean Filters
4. Contra Harmonic Mean Filters

2. Order Statistics Filters

1. Median Filters
2. Min & Max Filters
3. Mid Point Filters
4. Alpha Trimmed Mean Filters



Handwritten equations showing the degradation model and its Fourier transform:

$$g(x, y) = f(x, y) + n(x, y)$$
$$\text{or } G(u, v) = F(u, v) + N(u, v)$$

There are two types of spatial filtering methods that restore the image in the presence of noise only:

A. Mean Filters

1. Arithmetic Mean Filters
2. Geometric Mean Filters
3. Harmonic Mean Filters
4. Contra Harmonic Mean Filters

Mean filters reduce noise by replacing each pixel value with a value computed from its neighborhood. They are mainly used to smooth images and reduce random noise.

The **Arithmetic Mean Filter** replaces the center pixel by the **average of all pixels** in a window (mask).

Formula

For an $m \times n$ window:

1. Arithmetic Mean Filter

$$\hat{f}(x, y) = \frac{1}{mn} \sum_{(s,t) \in S_{xy}} g(s, t)$$

Where:

- $g(s, t)$ = noisy image
- S_{xy} = neighborhood window

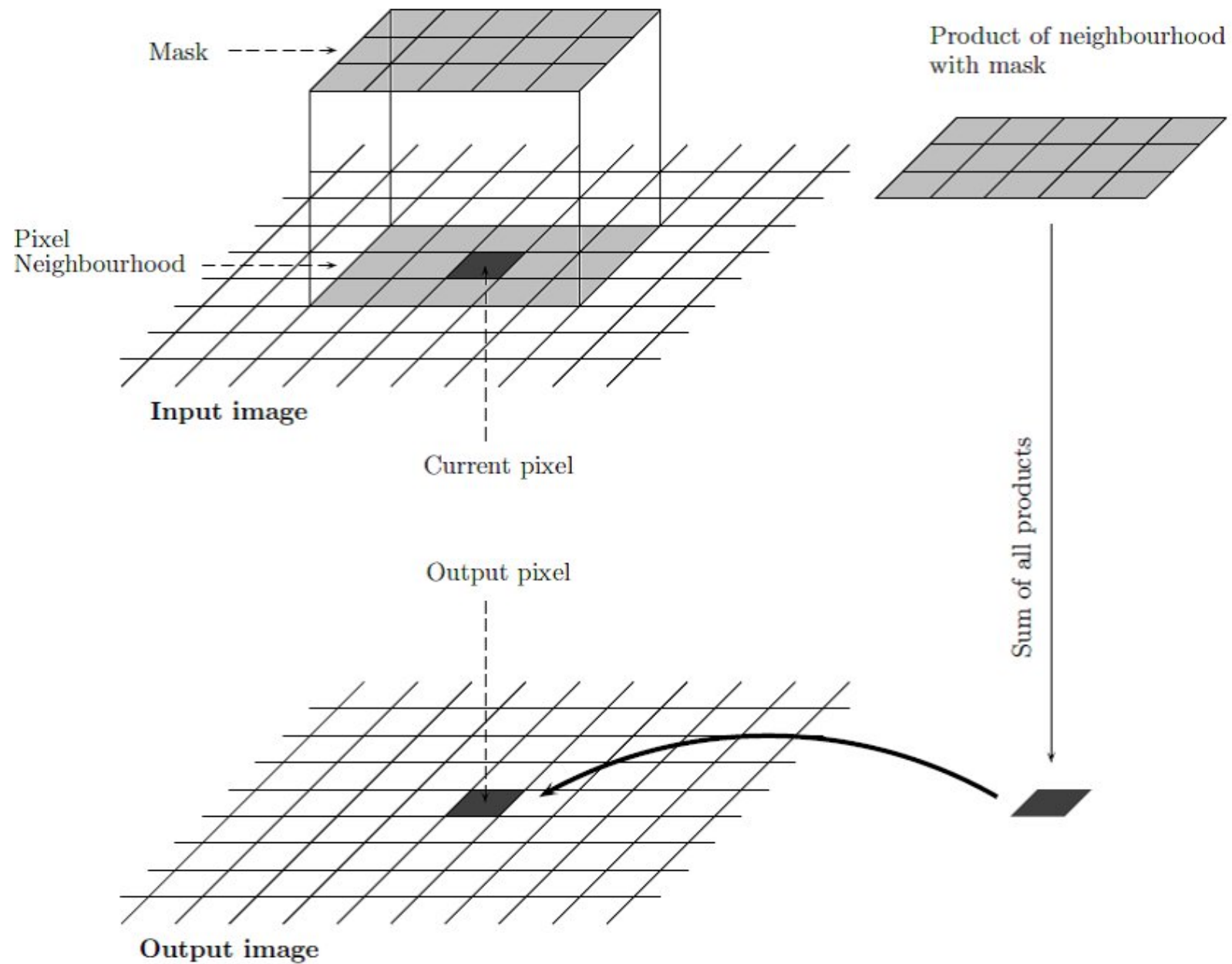


Figure 3.2: Performing spatial filtering

Example (3×3 window)

$$\begin{bmatrix} 10 & 12 & 11 \\ 13 & 15 & 14 \\ 12 & 11 & 10 \end{bmatrix}$$

Sum = 108

$$\text{Output} = \frac{108}{9} = 12$$

Characteristics

- Smooths the image
- Blurs edges
- Best for **Gaussian / uniform noise**

2. Geometric Mean Filter

The **Geometric Mean Filter** multiplies all pixel values and then takes the **nth root**.

Formula

$$\hat{f}(x, y) = \left(\prod_{(s,t) \in S_{xy}} g(s, t) \right)^{\frac{1}{mn}}$$

Example (3×3 window)

Values:

$\{10, 12, 11, 13, 15, 14, 12, 11, 10\}$

Product = P

$$\hat{f} = P^{1/9} \approx 11.9$$

Characteristics

- Less blurring than arithmetic mean
- Preserves detail better
- Useful for **multiplicative noise (speckle)**

Original
Image

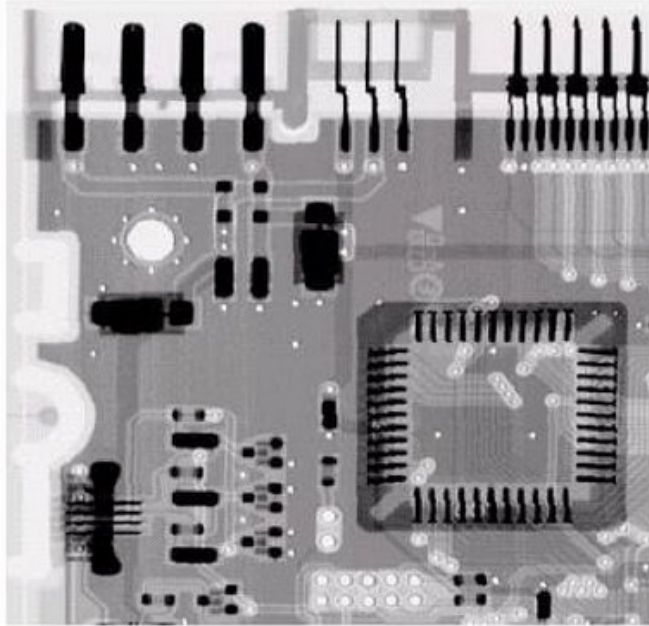
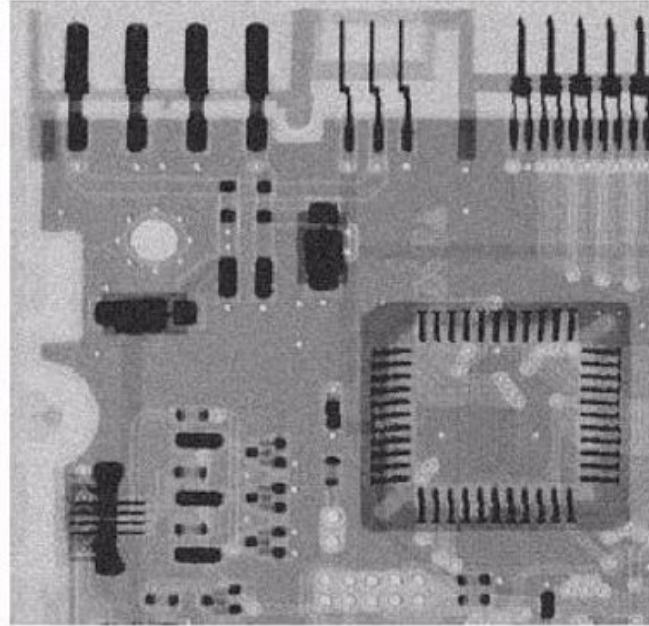
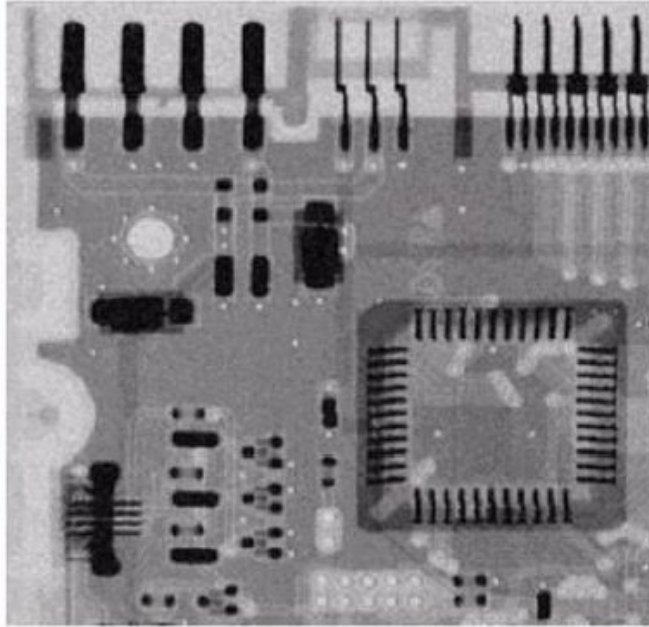


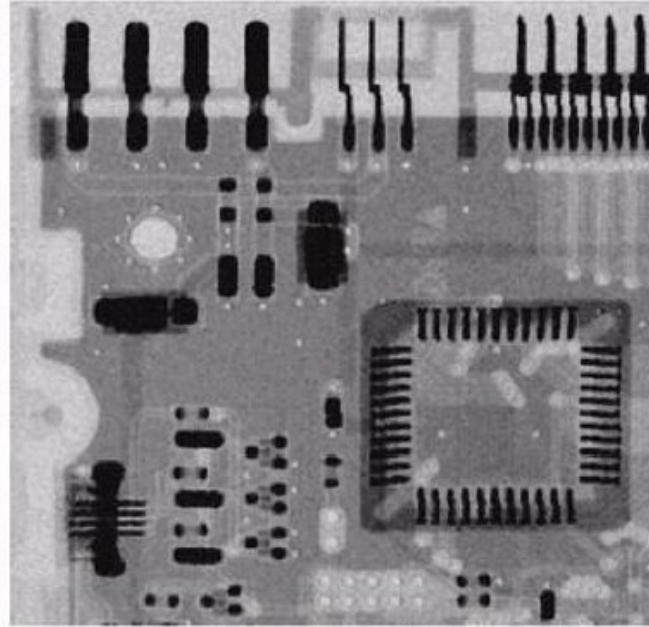
Image
Corrupted
By Gaussian
Noise



After A 3*3
Arithmetic
Mean Filter



After A 3*3
Geometric
Mean Filter



The **Harmonic Mean Filter** uses the reciprocal of pixel values.

Formula

3. Harmonic Mean Filter

$$\hat{f}(x, y) = \frac{mn}{\sum_{(s,t) \in S_{xy}} \frac{1}{g(s,t)}}$$

$$\begin{bmatrix} 10 & 12 & 11 \\ 13 & 15 & 14 \\ 12 & 11 & 10 \end{bmatrix}$$

$$\sum \frac{1}{g(s,t)} \approx 0.78$$

$$\hat{f} = \frac{9}{0.78} \approx 11.5$$

$$H = \frac{n}{\frac{1}{x_1} + \frac{1}{x_2} + \dots + \frac{1}{x_n}}$$

Characteristics

- Effective for **salt noise** (high intensity spikes)
- Fails for pepper noise (zeros)

4. Contra-Harmonic Mean Filter

An extension of the harmonic mean filter with a **parameter Q** that controls noise removal.

Formula

$$\hat{f}(x, y) = \frac{\sum g(s, t)^{Q+1}}{\sum g(s, t)^Q}$$

Effect of Q

- $Q > 0 \rightarrow$ removes **pepper noise** (dark spots)
- $Q < 0 \rightarrow$ removes **salt noise** (bright spots)

Example

If pepper noise is present, choose $Q = +1$

If salt noise is present, choose $Q = -1$

Characteristics

- Very powerful for **salt-and-pepper noise**
- Cannot remove both simultaneously
- Sensitive to zeros (when $Q < 0$)

Consider the following pixel values:

$$\begin{bmatrix} 10 & 10 & 10 \\ 10 & 0 & 10 \\ 10 & 10 & 10 \end{bmatrix}$$

Here, **0** represents **pepper noise**.

Case 1: Removing Pepper Noise

Choose $Q = +1$

Step 1: Calculate numerator

$$\begin{aligned} \sum g^{Q+1} &= \sum g^2 \\ &= (10^2 \times 8) + 0^2 = 800 \end{aligned}$$

Step 2: Calculate denominator

$$\begin{aligned} \sum g^Q &= \sum g \\ &= (10 \times 8) + 0 = 80 \end{aligned}$$

Formula

$$\hat{f}(x, y) = \frac{\sum g(s, t)^{Q+1}}{\sum g(s, t)^Q}$$

Where:

- $g(s, t)$ = pixel values in the neighborhood
- Q = order of the filter

Step 3: Compute output pixel

$$\hat{f}(x, y) = \frac{800}{80} = \boxed{10}$$

✅ Pepper noise is removed successfully.

Case 2: Removing Salt Noise

Choose $Q = -1$

Consider salt noise example:

$$\begin{bmatrix} 10 & 10 & 10 \\ 10 & 255 & 10 \\ 10 & 10 & 10 \end{bmatrix}$$

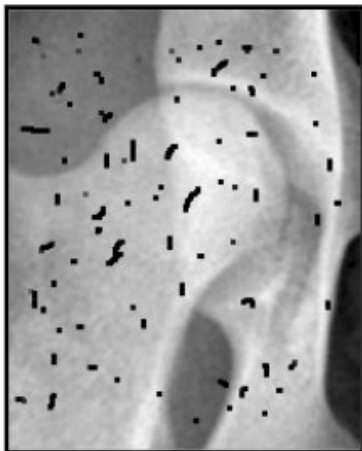
(High value **255** = salt noise)

Applying $Q = -1$ suppresses the high value and restores the pixel near **10**.

Q value	Noise Removed
$Q > 0$	Pepper noise
$Q < 0$	Salt noise
$Q = 0$	Arithmetic mean filter

Before:

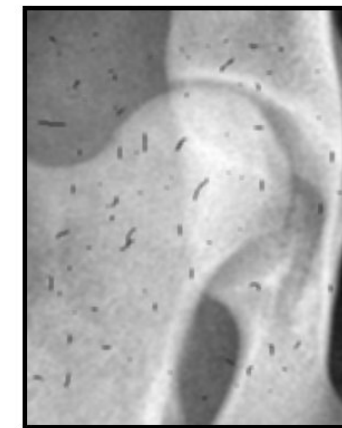
After:



Before:

After (Q=1):

After (Q=-1):



Filter Type	Best For	Edge Preservation	Noise Type
Arithmetic Mean	General smoothing	Poor	Gaussian
Geometric Mean	Detail preservation	Better	Speckle
Harmonic Mean	Salt noise	Moderate	Salt
Contra-Harmonic	Salt or Pepper (controlled)	Good	Impulse

Order Statistics Filters (Non-Linear Spatial Filters)

Order statistics filters work by **sorting pixel values** inside a neighborhood and then selecting or computing values from the ordered list.

They are **non-linear filters** and are very effective for **impulse (salt-and-pepper) noise**.

1. Median Filter

The **Median Filter** replaces the center pixel with the **median (middle) value** of the sorted neighborhood.

Formula

$$\hat{f}(x, y) = \text{median}\{g(s, t)\}$$

Example (3×3 window)

$$\begin{bmatrix} 10 & 255 & 12 \\ 11 & 9 & 10 \\ 8 & 10 & 11 \end{bmatrix}$$

Sorted values:

$$\{8, 9, 10, 10, 10, 11, 11, 12, 255\}$$

Median = 10

Characteristics

- Excellent for **salt-and-pepper noise**
- Preserves edges better than mean filters
- Non-linear filter

2. Min and Max Filters

(a) Min Filter

Definition

Replaces the pixel with the **minimum value** in the neighborhood.

Formula

$$\hat{f}(x, y) = \min\{g(s, t)\}$$

Use

- Removes **salt noise (bright spots)**
- Darkens the image

(b) Max Filter

Definition

Replaces the pixel with the **maximum value** in the neighborhood.

Formula

$$\hat{f}(x, y) = \max\{g(s, t)\}$$

Use

- Removes **pepper noise (dark spots)**
- Brightens the image

3. Midpoint Filter

The **Midpoint Filter** uses the average of the **maximum and minimum** values in the window.

Formula

$$\hat{f}(x, y) = \frac{1}{2} [\max\{g(s, t)\} + \min\{g(s, t)\}]$$

Neighborhood values:

$$\{10, 12, 14, 15, 18, 20, 22, 24, 25\}$$

Min = 10

Max = 25

$$\hat{f} = \frac{10 + 25}{2} = 17.5$$

Characteristics

- Effective for **uniform and Gaussian noise**
- Not effective for impulse noise

4. Alpha-Trimmed Mean Filter

The **Alpha-Trimmed Mean Filter** removes the $d/2$ lowest and $d/2$ highest pixel values, then computes the mean of the remaining pixels.

Formula

$$\hat{f}(x, y) = \frac{1}{mn - d} \sum g_r(s, t)$$

Sorted values:

$\{2, 4, 6, 8, 10, 12, 14, 200, 255\}$

Where:

- d = number of discarded pixels
- g_r = remaining pixels after trimming

Let $d = 4$

Remove $\rightarrow \{2, 4\}$ and $\{200, 255\}$

Remaining:

$\{6, 8, 10, 12, 14\}$

Characteristics

- Combines **mean and median filter properties**
- Effective for **mixed Gaussian + impulse noise**
- Flexible by adjusting d

Mean =

$$\frac{6 + 8 + 10 + 12 + 14}{5} = 10$$

Filter	Operation	Best for Noise	Edge Preservation
Median	Middle value	Salt & Pepper	Excellent
Min	Minimum value	Salt	Poor
Max	Maximum value	Pepper	Poor
Midpoint	$(\text{Max} + \text{Min})/2$	Gaussian	Moderate
Alpha-Trimmed	Mean after trimming	Mixed noise	Good

Frequency Domain Filters

Band-Reject and Band-Pass Filters

In frequency domain image processing, filters work on the **Fourier spectrum** of an image.

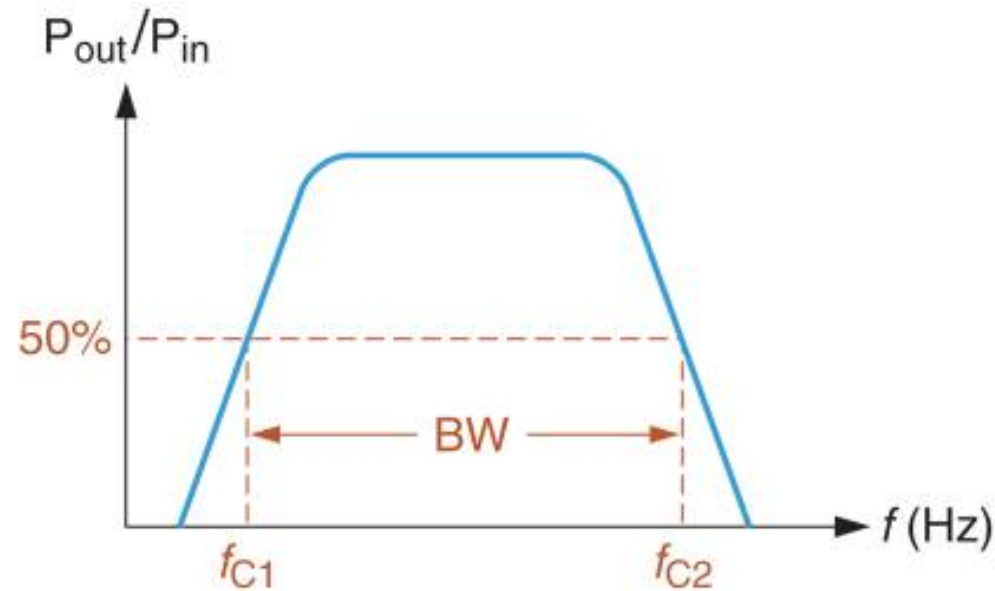
They selectively remove or retain frequencies based on their distance from the **origin (center)**.

- **Band-Reject Filter:**

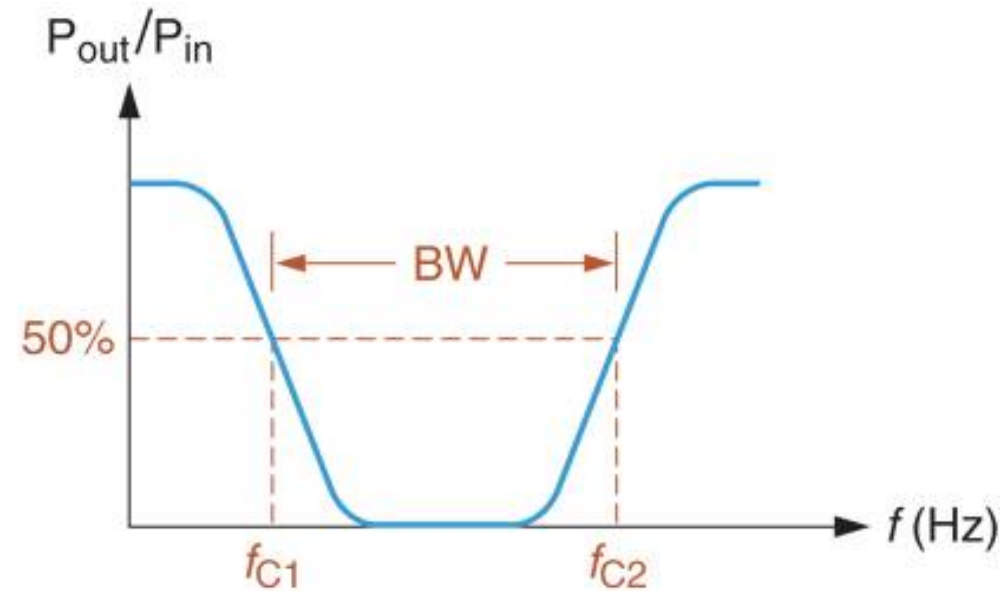
Removes frequencies within a specified band to eliminate periodic noise.

- **Band-Pass Filter:**

Allows only a selected frequency band to pass, emphasizing textures and details.



(a) Bandpass



(b) Band-stop (notch)

1. Band-Reject Filter (BRF)

A **Band-Reject Filter** blocks (rejects) a **specific range of frequencies** and allows frequencies **below and above** that band to pass.

✚ Also called **Band-Stop Filter** or **Notch Filter**.

Ideal Band-Reject Filter Formula

$$H(u, v) = \begin{cases} 0, & D_0 - \frac{W}{2} \leq D(u, v) \leq D_0 + \frac{W}{2} \\ 1, & \text{otherwise} \end{cases}$$

Use Case

- *Removing periodic interference (horizontal/vertical stripes)*
- *Removing power-line noise*

Where:

- $D(u, v)$ = distance from frequency origin
- D_0 = center frequency
- W = bandwidth

2. Band-Pass Filter (BPF)

A **Band-Pass Filter** allows **only a specific range of frequencies** to pass and blocks all others.

✚ It is the **inverse of a band-reject filter**.

Use Case

- Texture analysis
- Edge enhancement
- Feature extraction

$$H(u, v) = \begin{cases} 1, & D_0 - \frac{W}{2} \leq D(u, v) \leq D_0 + \frac{W}{2} \\ 0, & \text{otherwise} \end{cases}$$

Relationship Between BPF and BRF

$$\text{Band-Pass Filter} = 1 - \text{Band-Reject Filter}$$

Image Compression

Compression is the process of reducing the size of data (like images, audio, video, or text) so that it takes up less storage space or can be transmitted faster over networks.

- It works by removing redundancy or unnecessary information in the data.
- Compressed data can either be restored to its original form (**lossless**) or approximated (**lossy**).



Types of Compression

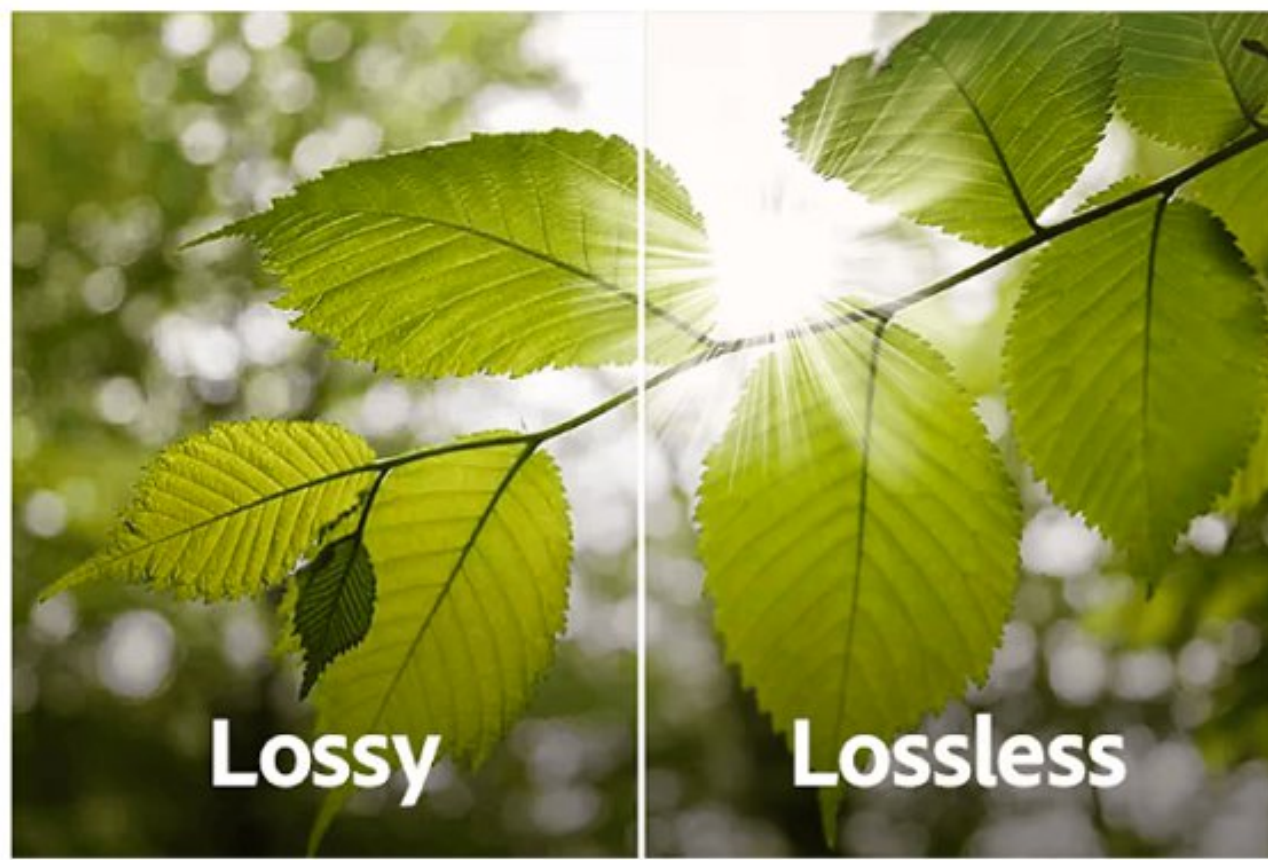
A. Lossless Compression

- **Definition:** Compression in which the original data can be perfectly restored after decompression. No information is lost.
- **How it works:** Uses techniques like finding repeating patterns, encoding, or statistical methods.
- **Examples in images and files:**
 - **Text files:** ZIP, GZIP
 - **Images:** PNG, BMP (compressed with RLE or Huffman coding)
 - **Audio:** FLAC
- **Use case:** When exact original data is needed, like text documents, source code, or medical images.

B. Lossy Compression

- **Definition:** Compression in which some information is permanently lost, resulting in approximate reconstruction of the original data.
- **How it works:** Removes details that are less noticeable to humans (like minor color differences in images or sound frequencies in audio).
- **Examples in images and files:**
 - **Images:** JPEG
 - **Audio:** MP3, AAC
 - **Video:** MPEG, MP4
- **Use case:** When some loss of quality is acceptable for significant reduction in size, like streaming video, music, or web images.

Feature	Lossless Compression	Lossy Compression
Data recovery	Perfectly restored	Partially restored
Quality after decompression	Original quality	Slightly reduced
File size reduction	Moderate	High
Examples	PNG, ZIP, FLAC	JPEG, MP3, MP4



Key Concepts in Image Compression

1. Compression Ratio

The **compression ratio** is a measure of the effectiveness of a compression algorithm. It is defined as the ratio of the original image size to the compressed image size.

$$\text{Compression Ratio} = \frac{\text{Original Image Size}}{\text{Compressed Image Size}}$$

For example, if an original image is 10 MB and the compressed image is 2 MB, the compression ratio is 5:1.

2. Average Length of Code

The **average length of code** refers to the average number of bits used to represent each symbol (e.g., pixel value) in the compressed image. This concept is particularly relevant in lossless compression techniques that use variable-length coding schemes, such as Huffman coding.

The average length of code can be calculated using the formula:

$$L_{\text{avg}} = \sum_{i=1}^n P_i \cdot l_i$$

where:

- P_i is the probability of occurrence of the i -th symbol.
- l_i is the length of the codeword for the i -th symbol.
- n is the total number of different symbols.

Example:

Consider an image with 4 pixel intensity values: A, B, C, D . Their probabilities and corresponding code lengths are as follows:

Symbol	Probability (p_i)	Code Length (l_i)
A	0.5	1 bit
B	0.25	2 bits
C	0.125	3 bits
D	0.125	3 bits

Interpretation:

In this example, the average length of code is 1.75 bits per symbol. This means that, on average, each pixel in the compressed image requires 1.75 bits to represent it. A good compression algorithm aims to minimize this value while maintaining image quality.

The average length of code is calculated as:

$$L = (0.5 \times 1) + (0.25 \times 2) + (0.125 \times 3) + (0.125 \times 3)$$
$$L = 0.5 + 0.5 + 0.375 + 0.375 = 1.75 \text{ bits per symbol}$$

Huffman Coding for image compression

Huffman Coding is a lossless data compression technique used to reduce the size of data by assigning variable-length binary codes to symbols (characters). The key idea is simple:

**More frequent symbols get shorter codes,
Less frequent symbols get longer codes.**

It is widely used in image compression (JPEG), audio compression, and file formats (ZIP, PNG).

Basic Idea (Intuition)

- Fixed-length coding (like ASCII): every character uses the same number of bits.
- Huffman coding: uses **fewer bits overall** by exploiting symbol frequency.

Example:

- Frequently occurring character → short code (e.g., 0)
- Rare character → long code (e.g., 1101)

Steps of Huffman Coding Algorithm

1. Count frequency of each symbol.
2. Create a leaf node for each symbol.
3. Build a binary tree:
 - Pick two nodes with **lowest frequencies**
 - Merge them into a new node (sum of frequencies)
4. Repeat until only one node (root) remains.
5. Assign binary codes:
 - Left edge → 0
 - Right edge → 1
6. Codes obtained from root to leaf give the **Huffman codes**.

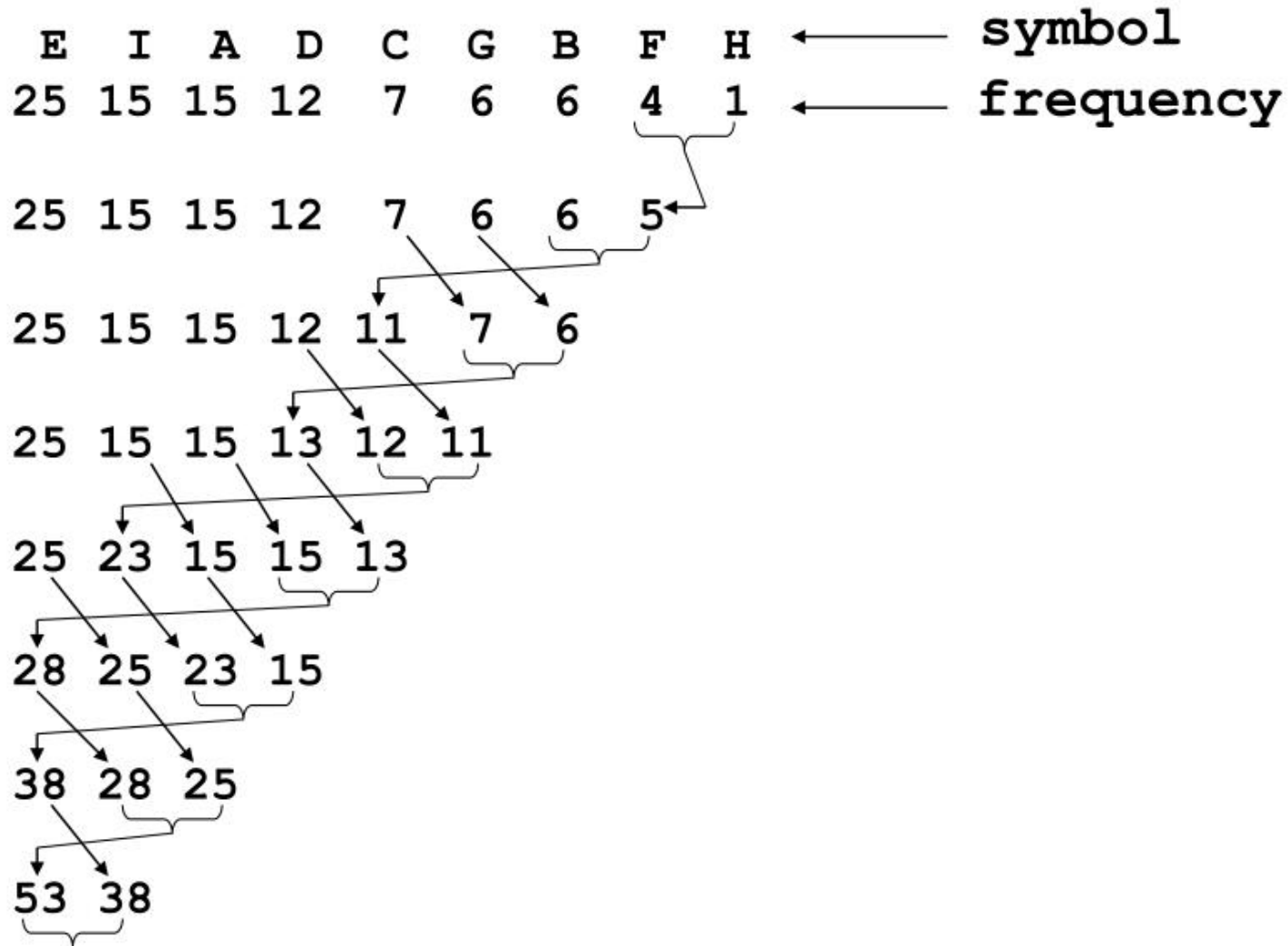
Given Symbols and Frequencies

Symbol	Frequency
A	5
B	9
C	12
D	13
E	16
F	45

◆ Rule to Remember

- Always pick the TWO symbols with the LOWEST frequencies
- Combine them
- Repeat until only **one node (root)** remains

Construction of a Huffman tree



Symbol	Frequency
A	5
B	9
C	12
D	13
E	16
F	45

◆ Step 1: Pick two smallest

Smallest frequencies:

- **A (5) and B (9)**

Combine:

CSS



◆ Step 2: Pick next two smallest

Remaining nodes:

- (14), C(12), D(13), E(16), F(45)

Smallest:

- **C (12) and D (13)**

Combine:

mathematica



◆ Step 3: Pick next two smallest

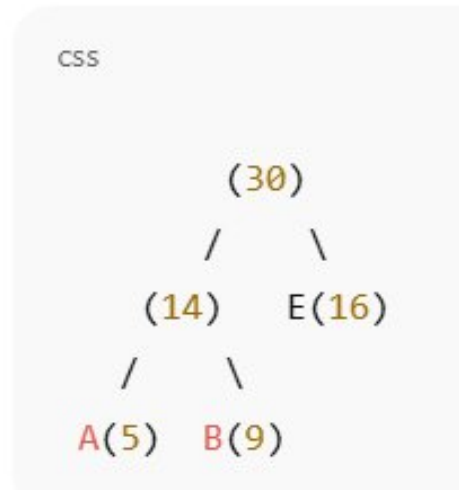
Remaining nodes:

- (14), (25), E(16), F(45)

Smallest:

- (14) and E (16)

Combine:



◆ Step 4: Pick next two smallest

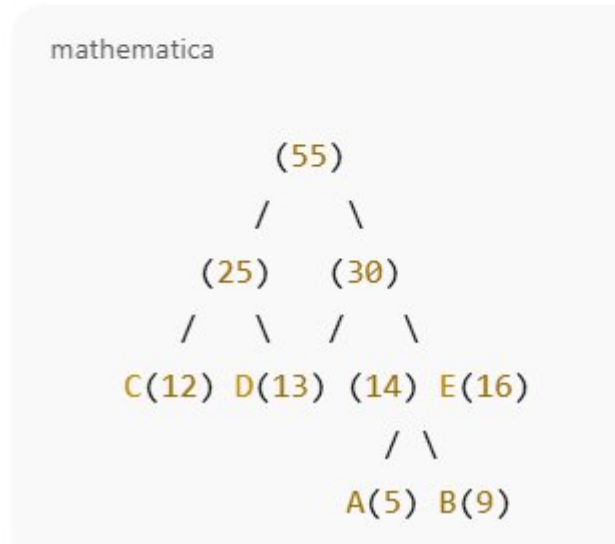
Remaining nodes:

- (25), (30), F(45)

Smallest:

- (25) and (30)

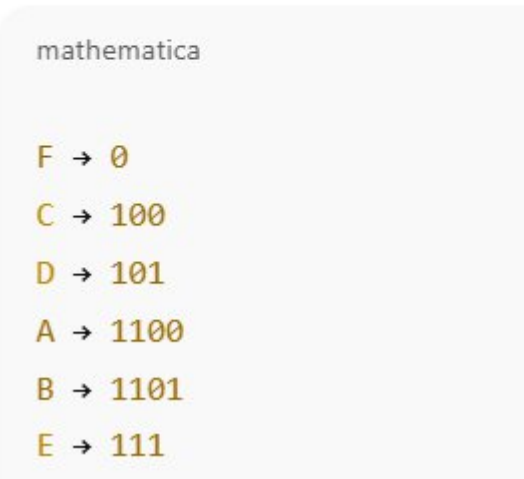
Combine:



◆ Assign Binary Codes

Rule:

- Left edge → 0
- Right edge → 1

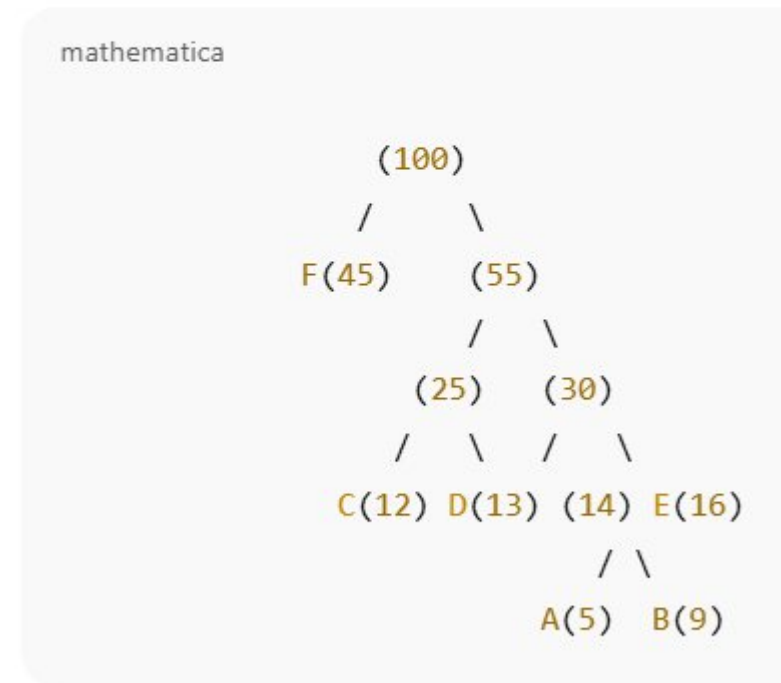


◆ Step 5: Final Combination

Remaining nodes:

- (55) and F (45)

Combine → ROOT



✓ Huffman Tree completed

Step 1: Given data (same as before)

Symbol	Frequency
A	5
B	9
C	12
D	13
E	16
F	45
Total symbols	100

◆ Case 1: Fixed-Length Coding (WITHOUT Huffman)

How many bits per symbol?

We have 6 symbols.

$$\text{Bits required} = \lceil \log_2 6 \rceil = 3 \text{ bits}$$

So every symbol uses **3 bits**, even if it appears many times.

Total bits used

$$100 \text{ symbols} \times 3 \text{ bits} = 300 \text{ bits}$$

◆ Case 2: Huffman Coding (WITH compression)

Huffman codes (one valid set)

Symbol	Code	Code Length
F	0	1
C	100	3
D	101	3
E	111	3
A	1100	4
B	1101	4

Multiply **frequency × code length**

Symbol	Frequency	Bits per symbol	Total bits
A	5	4	20
B	9	4	36
C	12	3	36
D	13	3	39
E	16	3	48
F	45	1	45
Total	100		224 bits

✓ Huffman coding uses **224 bits**

◆ Step 3: Compression Calculation

Bits saved

$$300 - 224 = 76 \text{ bits saved}$$

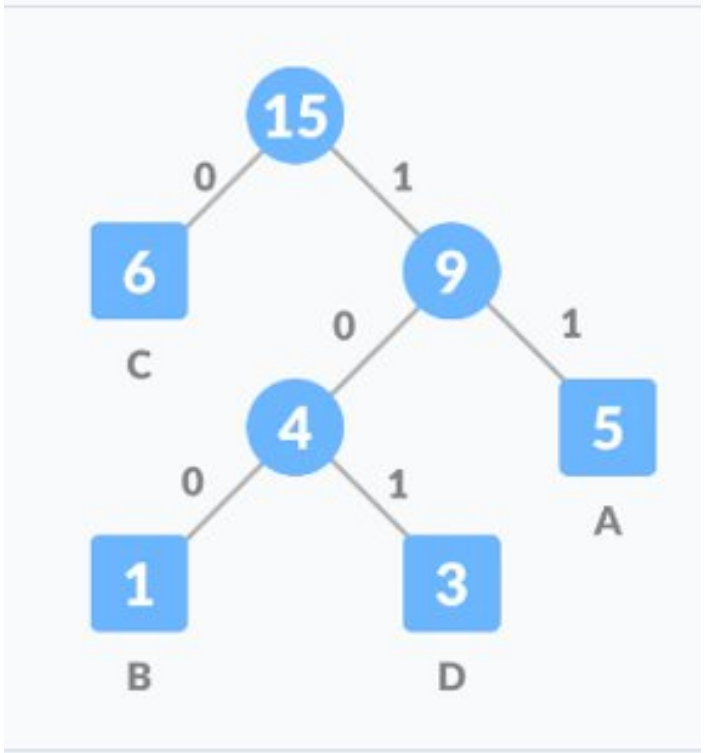
 That's “around 25% compression”

Huffman coding achieves compression by assigning shorter codes to frequently occurring symbols and longer codes to less frequent symbols, thereby reducing the average number of bits per symbol.

B C A A D D D C C A C A C A C

Character	Frequency	Code	Size
A	5	11	$5 \times 2 = 10$
B	1	100	$1 \times 3 = 3$
C	6	0	$6 \times 1 = 6$
D	3	101	$3 \times 3 = 9$
$4 \times 8 = 32$ bits	15 bits		28 bits

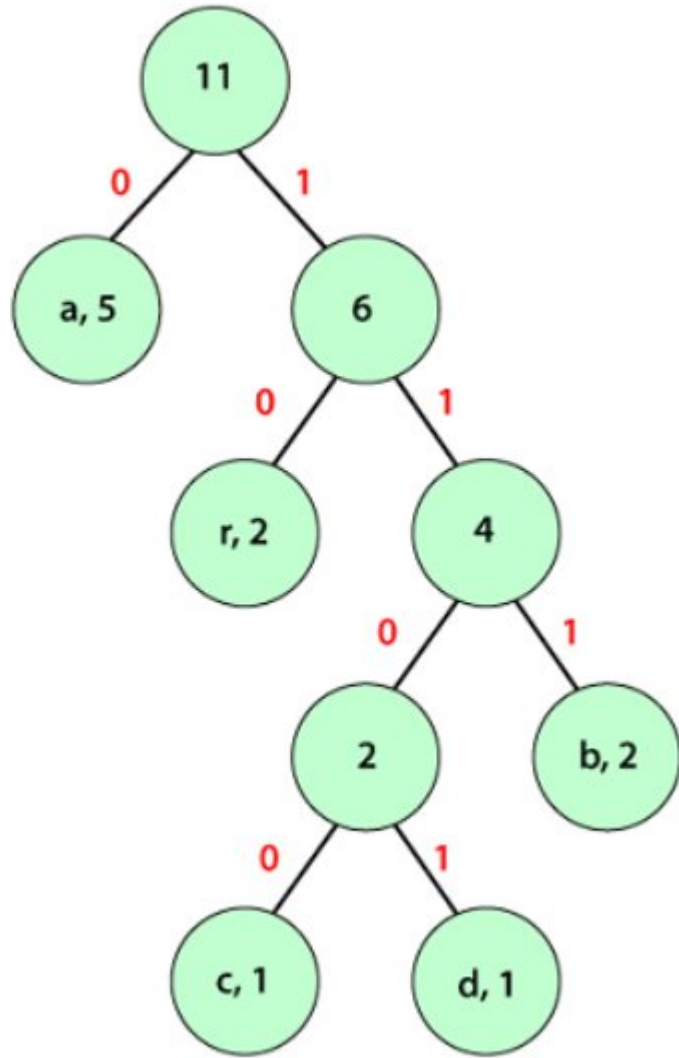
Without encoding, the total size of the string was 120 bits. After encoding the size is reduced to $32 + 15 + 28 = 75$.



Huffman Encoding Example

Suppose, we have to encode string **abracadabra**. Determine the following:

- i. Huffman code for All the characters
- ii. Average code length for the given String
- iii. Length of the encoded string



Character	Frequency	Code	Code Length
A	5	0	1
B	2	111	3
C	1	1100	4
D	1	1101	4
R	2	10	2

Now we can encode the string **(abracadabra)** that we have taken above.

0 111 10 0 1100 0 1101 0 111 10 0

(ii) Average Code Length for the String

The average code length of the Huffman tree can be determined by using the formula given below:

$$\text{Average Code Length} = \sum (\text{frequency} \times \text{code length}) / \sum (\text{frequency})$$

$$= \{ (5 \times 1) + (2 \times 3) + (1 \times 4) + (1 \times 4) + (2 \times 2) \} / (5+2+1+1+2)$$

$$= \mathbf{2.09090909}$$

(iii) Length of the Encoded String

The length of the encoded message can be determined by using the following formula:

$$\text{length} = \text{Total number of characters in the text} \times \text{Average code length per character}$$

$$= 11 \times 2.09090909$$

$$= \mathbf{23 \text{ bits}}$$

Construct Huffman code for each gray level given and find the compression ratio and coding efficiency Gray level (0,1,2,4,5,6,7) and No of Pixel (30,35,38,15,10,38,80)

◆ Step 1: Total number of pixels

$$30 + 35 + 38 + 15 + 10 + 38 + 80 = 246$$

Gray level	No. of pixels
0	30
1	35
2	38
4	15
5	10
6	38
7	80

Gray level	Pixels	Probability
0	30	0.122
1	35	0.142
2	38	0.154
4	15	0.061
5	10	0.041
6	38	0.154
7	80	0.325

- ◆ Step 1: Sort by frequency (ascending)

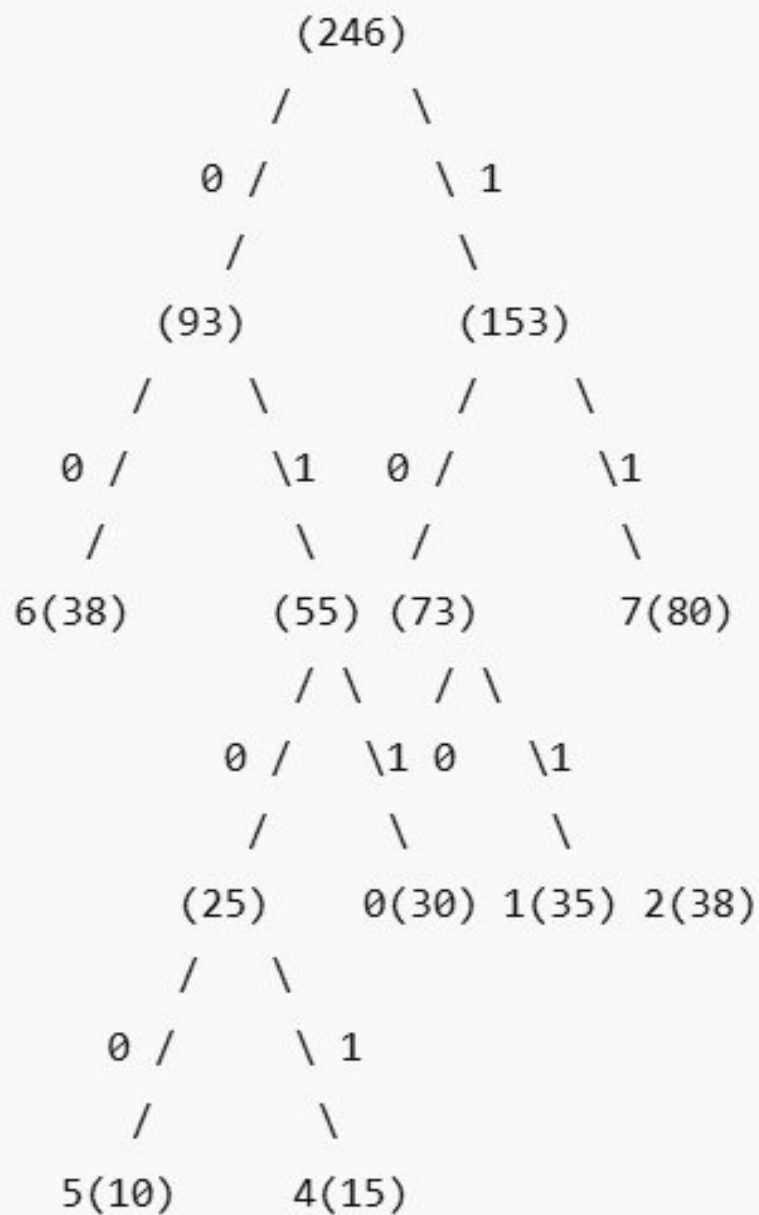
SCSS

5(10), 4(15), 0(30), 1(35), 2(38), 6(38), 7(80)

Combination steps

1. $5(10) + 4(15) \rightarrow 25$
2. $25 + 0(30) \rightarrow 55$
3. $1(35) + 2(38) \rightarrow 73$
4. $6(38) + 55 \rightarrow 93$
5. $73 + 7(80) \rightarrow 153$
6. $93 + 153 \rightarrow 246$ (Root)

Gray level	Huffman Code
6	00
5	0100
4	0101
0	011
1	100
2	101
7	11



◆ Step 4: Assign Huffman Codes

(Left = 0, Right = 1)

Gray level	Huffman Code	Code Length
6	00	2
5	0100	4
4	0101	4
0	011	3
1	100	3
2	101	3
7	11	2

◆ Step 5: Average Code Length (L)

$$L = \sum P(i) \times l(i)$$

Gray level	P(i)	l(i)	P×l
0	0.122	3	0.366
1	0.142	3	0.426
2	0.154	3	0.462
4	0.061	4	0.244
5	0.041	4	0.164
6	0.154	2	0.308
7	0.325	2	0.650
Total L			2.62 bits/pixel

◆ Step 6: Entropy (H)

$$H = - \sum P(i) \log_2 P(i)$$

Calculated value:

$$H \approx 2.53 \text{ bits/pixel}$$

Entropy in Huffman coding is the average minimum number of bits required to represent a symbol, based on its probability of occurrence.

◆ Step 7: Compression Ratio (CR)

Fixed-length coding:

7 gray levels →

$$\lceil \log_2 7 \rceil = 3 \text{ bits/pixel}$$

Huffman coding:

$$L = 2.62 \text{ bits/pixel}$$

$$CR = \frac{3}{2.62} = 1.145$$

✅ Compression Ratio $\approx 1.15 : 1$

◆ Step 8: Coding Efficiency (η)

$$\eta = \frac{H}{L} \times 100$$

$$\eta = \frac{2.53}{2.62} \times 100 = 96.6\%$$

✅ Final Answers

- Huffman average code length = 2.62 bits/pixel
- Entropy = 2.53 bits/pixel
- Compression ratio = 1.15 : 1
- Coding efficiency = 96.6%

Interpixel Redundancy (Run Length Coding)

Run-Length Coding (RLC) is a simple and efficient **lossless compression** technique used to reduce the size of data by replacing consecutive repeated elements (runs) with a single value and a count of its repetitions. It is particularly effective for data with long sequences of repeated values, such as binary images, grayscale images, or text files.



Run-length coding (RLC) exploits the repetitive nature of the image

Example: Given a sample binary image. Apply Run-length Coding.

Solutions → ① Horizontal RLC

* Run Length vectors ⇒ (0,5)

* No. of vectors = 6 (0,3), (1,2)

* Max. length = 5
↓
3 bits in binary (1,5)
(1,5)
(1,5)

* No. of bits per pixel = 1 (0 or 1)

* Total no. of pixels = $6 \times (3+1)$
 $= 24$ ✓

* No. of pixels for original image = $5 \times 5 = 25$ ✓

* Compression Ratio = $\frac{25}{24} = 1.042:1$
=

* →

0	0	0	0	0
0	0	0	1	1
1	1	1	1	1
1	1	1	1	1
1	1	1	1	1

5×5

Apply Run-length Coding.

② Vertical RLC

* Run length vectors

✓ (0, 2), (1, 3)

✓ (0, 2), (1, 3)

(0, 2), (1, 3)

(0, 1), (1, 4)

(0, 1), (1, 4)

* No. of vectors = 10

* 3 bits

* 1 bit per pixel

* Total no. of pixels $\Rightarrow$
 $= 10 \times (3 + 1) = 40$ ✓

* original Image = $5 \times 5 = 25$

* compression Ratio = $\frac{25}{40} = 0.625$ 

↓

0	0	0	0	0
0	0	0	1	1
1	1	1	1	1
1	1	1	1	1
1	1	1	1	1

5×5

Benefits of Run Length Encoding

1. **Compression Efficiency:** Effective for images with large regions of uniform color.
2. **Simplicity:** Easy to implement and understand.
3. **Lossless:** No loss of data during compression.

Psychovisual Redundancy (IGS Coding)

IGS coding is a technique used for image compression, particularly for grayscale images. It works by quantizing the pixel values to reduce the number of bits required to represent them, while preserving the visual quality of the image.

The key idea is to use a **bit-plane slicing** approach, where the most significant bits (MSBs) are retained, and the least significant bits (LSBs) are discarded or compressed.

How IGS Coding Works

1. Bit-Plane Slicing:

- A grayscale image is represented using 8 bits per pixel (values from 0 to 255).
- The 8-bit pixel value is divided into two parts:
 - **Most Significant Bits (MSBs):** The higher-order bits (e.g., the first 4 bits).
 - **Least Significant Bits (LSBs):** The lower-order bits (e.g., the last 4 bits).

2. Quantization:

- The MSBs are retained as they carry most of the visual information.
- The LSBs are discarded or compressed, reducing the total number of bits required to represent the image.

3. Reconstruction:

- During decompression, the discarded LSBs are approximated (e.g., set to 0 or interpolated), and the image is reconstructed using the retained MSBs.

Example of IGS Compression

Step 1: Original Image

Consider a small grayscale image with the following pixel values (8-bit representation):

Pixel	Value (8-bit)
1	150
2	200
3	100
4	50

Step 2: Bit-Plane Slicing

Split each pixel value into MSBs (first 4 bits) and LSBs (last 4 bits):

Pixel	Value (8-bit)	MSBs (First 4 bits)	LSBs (Last 4 bits)
1	150	1001	0110
2	200	1100	1000
3	100	0110	0100
4	50	0011	0010

Step 3: Quantization

Retain only the MSBs and discard the LSBs:

Pixel	MSBs (First 4 bits)
1	1001
2	1100
3	0110
4	0011

Step 4: Compressed Data

The compressed data now uses only 4 bits per pixel instead of 8 bits. This reduces the storage requirement by 50%.

Step 5: Decompression

During decompression, the LSBs are approximated (e.g., set to 0). The reconstructed pixel values are:

Pixel	MSBs (First 4 bits)	Reconstructed Value (8-bit)
1	1001	10010000 = 144
2	1100	11000000 = 192
3	0110	01100000 = 96
4	0011	00110000 = 48

Compression Ratio

- **Original Size:** 8 bits/pixel × 4 pixels = 32 bits.
- **Compressed Size:** 4 bits/pixel × 4 pixels = 16 bits.
- **Compression Ratio:**

$$\text{Compression Ratio} = \frac{\text{Original Size}}{\text{Compressed Size}} = \frac{32}{16} = 2$$

Advantages of IGS Coding

1. Simple and computationally efficient.
2. Reduces storage requirements significantly.
3. Preserves most of the visual information by retaining the MSBs.

Limitations of IGS Coding

1. Loss of fine details due to discarding LSBs.
2. May introduce visible artifacts in the reconstructed image, especially in regions with smooth gradients.

Why IGS is Lossy

- In lossless compression, the original data can be perfectly reconstructed from the compressed data. However, in IGS coding:
 - The discarded LSBs cannot be recovered exactly.
 - The reconstructed image is an approximation of the original, not an exact copy.
- Original value: 150 (binary: 10010110).
- After IGS coding (retaining only the first 4 MSBs): 1001 (binary: 10010000 = 144 in decimal).
- The original value 150 cannot be recovered from 144 .

Lossless predictive coding model

- *Lossless Predictive Coding is an image compression technique in which each pixel is predicted from its neighboring pixels, and only the prediction error (residual) is encoded.*
- *Since the prediction error is encoded without any quantization, the original image can be perfectly reconstructed—hence it is lossless.*

Basic Idea

Instead of coding the actual pixel value $f(x, y)$, we code:

$$e(x, y) = f(x, y) - \hat{f}(x, y)$$

where

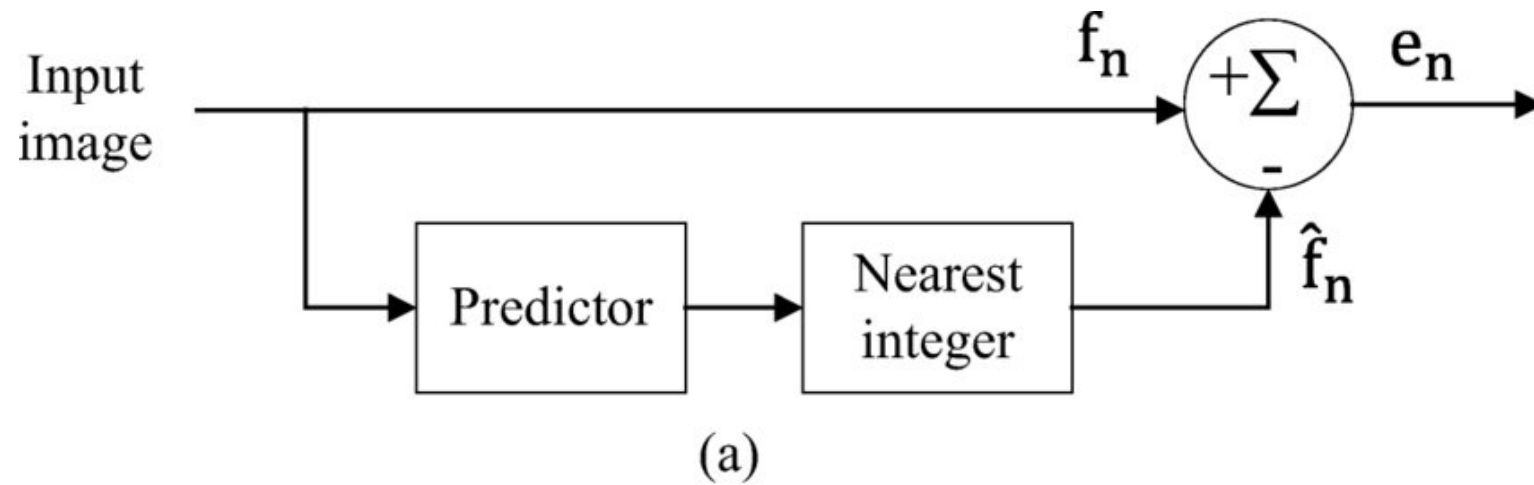
- $f(x, y)$ = actual pixel value
- $\hat{f}(x, y)$ = predicted pixel value
- $e(x, y)$ = prediction error

At the decoder:

$$f(x, y) = \hat{f}(x, y) + e(x, y)$$

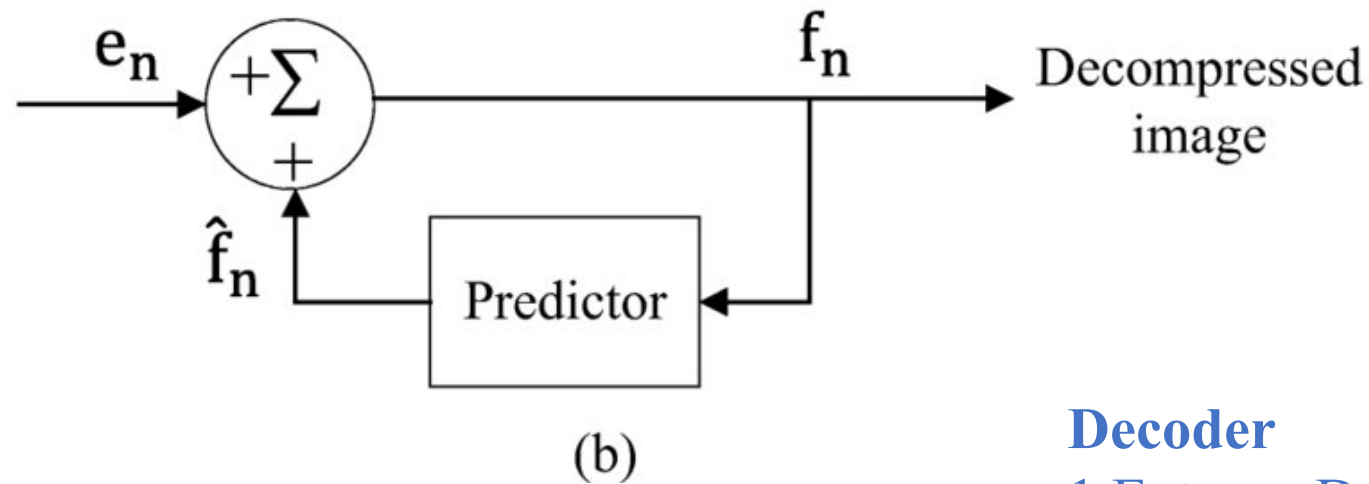
Applications

- Medical imaging (X-ray, MRI)
- Satellite and remote sensing images
- Archival image storage
- Standards like JPEG-LS, PNG (filtering stage)



Encoder

1. Predictor estimates the current pixel using past pixels.
2. Subtractor finds prediction error.
3. Entropy Encoder (Huffman / Arithmetic) compresses the error.



Decoder

1. Entropy Decoder decodes the error.
2. Predictor generates the same prediction.
3. Adder reconstructs the exact pixel value.

Example of **lossless predictive compression** using a sequence of original pixel values: **100, 102, 105, 107, 110** using a simple predictor and encode the prediction errors.

Step 1: Prediction

We'll use a simple predictor where the predicted value for a pixel is the value of the previous pixel:

$$\hat{I}(x) = I(x - 1)$$

- For the first pixel, there is no previous pixel, so we assume the prediction error is the pixel value itself.

Step 2: Calculate Prediction Errors

Pixel Position (x)	Original Value $I(x)$	Predicted Value $\hat{I}(x)$	Prediction Error $e(x) = I(x) - \hat{I}(x)$
1	100	0 (no prediction)	100
2	102	100	2
3	105	102	3
4	107	105	2
5	110	107	3

Step 3: Encode Prediction Errors

The prediction errors are: **100, 2, 3, 2, 3**. These errors are typically smaller than the original pixel values and can be compressed efficiently using a lossless encoding scheme (e.g., Huffman coding or arithmetic coding).

Step 4: Decompression

To reconstruct the original pixel values, we use the same predictor and add the prediction errors to the predicted values:

Pixel Position (x)	Predicted Value $\hat{I}(x)$	Prediction Error $e(x)$	Reconstructed Value $I(x) =$ $\hat{I}(x) + e(x)$
1	0 (no prediction)	100	100
2	100	2	102
3	102	3	105
4	105	2	107
5	107	3	110

Summary

- **Original Pixel Values:** 100, 102, 105, 107, 110
- **Prediction Errors:** 100, 2, 3, 2, 3
- **Reconstructed Pixel Values:** 100, 102, 105, 107, 110 (identical to the original)

Lossy predictive coding model

- *Lossy Predictive Coding is an image compression technique in which prediction error is quantized before encoding.*
- *Because of quantization, some information is lost, so the reconstructed image is not exactly the same as the original.*

Basic Idea

Instead of coding the actual pixel value $f(x, y)$, we code a **quantized prediction error**:

$$e(x, y) = f(x, y) - \hat{f}(x, y)$$

$$\hat{e}(x, y) = Q\{e(x, y)\}$$

At the decoder:

$$\tilde{f}(x, y) = \hat{f}(x, y) + \hat{e}(x, y)$$

where

- $Q\{\cdot\}$ = quantizer
- $\tilde{f}(x, y)$ = reconstructed pixel (approximate)

Applications

- Image compression (early standards)
- Speech coding
- Video coding (as part of hybrid codecs)
- Basis for JPEG predictive modes and video codecs

Lossy predictive coding model

a
b

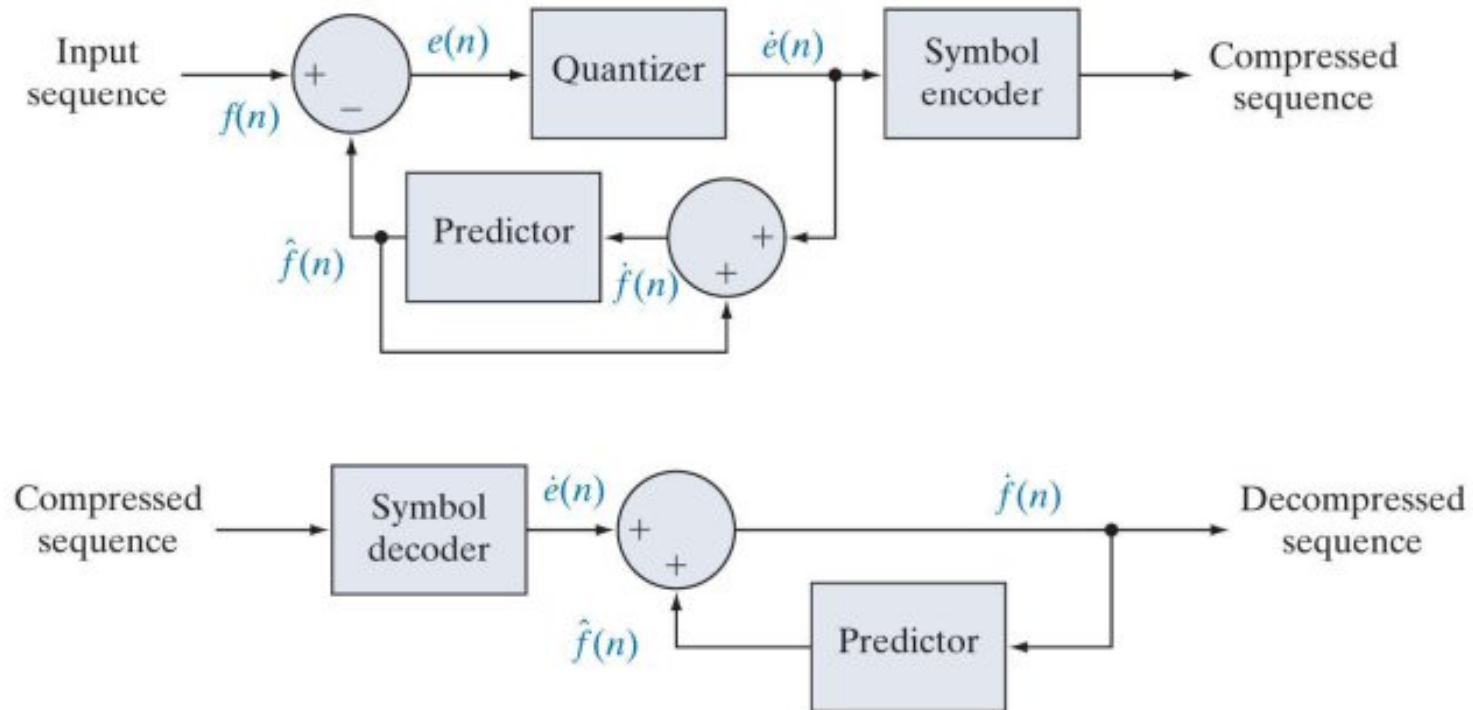


FIGURE 8.38

A lossy predictive coding model. (a) encoder; (b) decoder.

A **lossy predictive model** is a compression technique where the original data cannot be perfectly reconstructed after compression. Instead, some information is discarded to achieve higher compression ratios. This is done by quantizing the prediction errors, which introduces loss but reduces the amount of data to be stored or transmitted. Lossy compression is commonly used in applications where some loss of fidelity is acceptable, such as in images (JPEG), audio (MP3), and video (MPEG).

How It Works:

1. **Prediction:** A predictor estimates the value of the next data point based on previously observed data points.
2. **Error Calculation:** The difference between the actual value and the predicted value (prediction error) is computed.
3. **Quantization:** The prediction error is quantized (rounded to a smaller set of values), which introduces loss.
4. **Encoding:** The quantized prediction error is encoded using a lossless or lossy compression algorithm.
5. **Decoding:** During decompression, the quantized prediction error is added to the predicted value to reconstruct an approximation of the original data.

Using the same original pixel values: **100, 102, 105, 107, 110**, we'll apply a lossy predictive model. Assuming that we quantize the prediction errors by rounding them to the nearest multiple of 5.

Step 1: Prediction

We'll use the same predictor as before:

$$\hat{I}(x) = I(x - 1)$$

- For the first pixel, there is no previous pixel, so we assume the prediction error is the pixel value itself.

Step 2: Calculate Prediction Errors

Pixel Position (x)	Original Value $I(x)$	Predicted Value $\hat{I}(x)$	Prediction Error $e(x) = I(x) - \hat{I}(x)$
1	100	0 (no prediction)	100
2	102	100	2
3	105	102	3
4	107	105	2
5	110	107	3

Step 3: Quantize Prediction Errors

We quantize the prediction errors by rounding them to the nearest multiple of 5:

- $e(1) = 100 \rightarrow$ Quantized: 100
- $e(2) = 2 \rightarrow$ Quantized: 0
- $e(3) = 3 \rightarrow$ Quantized: 5
- $e(4) = 2 \rightarrow$ Quantized: 0
- $e(5) = 3 \rightarrow$ Quantized: 5

Quantized errors: **100, 0, 5, 0, 5**

Step 4: Encode Quantized Errors

The quantized errors are encoded using a lossless or lossy compression scheme. Since the errors are smaller and simpler, they can be compressed more efficiently.

Step 5: Decompression

To reconstruct the pixel values, we use the same predictor and add the quantized prediction errors to the predicted values:

Pixel Position (x)	Predicted Value $\hat{I}(x)$	Quantized Error $e_q(x)$	Reconstructed Value $I'(x) =$ $\hat{I}(x) + e_q(x)$
1	0 (no prediction)	100	100
2	100	0	100
3	100	5	105
4	105	0	105
5	105	5	110

Summary

- **Original Pixel Values:** 100, 102, 105, 107, 110
- **Quantized Errors:** 100, 0, 5, 0, 5
- **Reconstructed Pixel Values:** 100, 100, 105, 105, 110

Lab Questions

10. Add Gaussian noise to a given grayscale image and display the original and noisy images.
11. Add Salt and Pepper noise to a given image and observe its effect. Display both original and noisy images.

Construct Huffman code for each gray level. Calculate the compression ratio and the relative data redundancy assuming if 3-bit code is used to code the gray level instead of Huffman code. [7+3 =10]

7. Explain lossy predictive coding in brief.

9. Describe lossless predictive coding model with a suitable block diagram.

8. Explain Contra-harmonic Mean Filters used for image restoration.

3.Explain the contra-harmonic mean filters used in image restoration.

5. What do you mean by Lossy Predictive Coding? Explain it with a suitable block diagram.(6)

5. Construct Huffman code for each gray level given and find the compression ratio and coding efficiency.

Gray Level	0	1	2	4	5	6	7
No. of Pixels	30	35	38	15	10	38	80